

BUSINESS CENTRAL · ALDC

De asistentes de IA a ingeniería de agentes

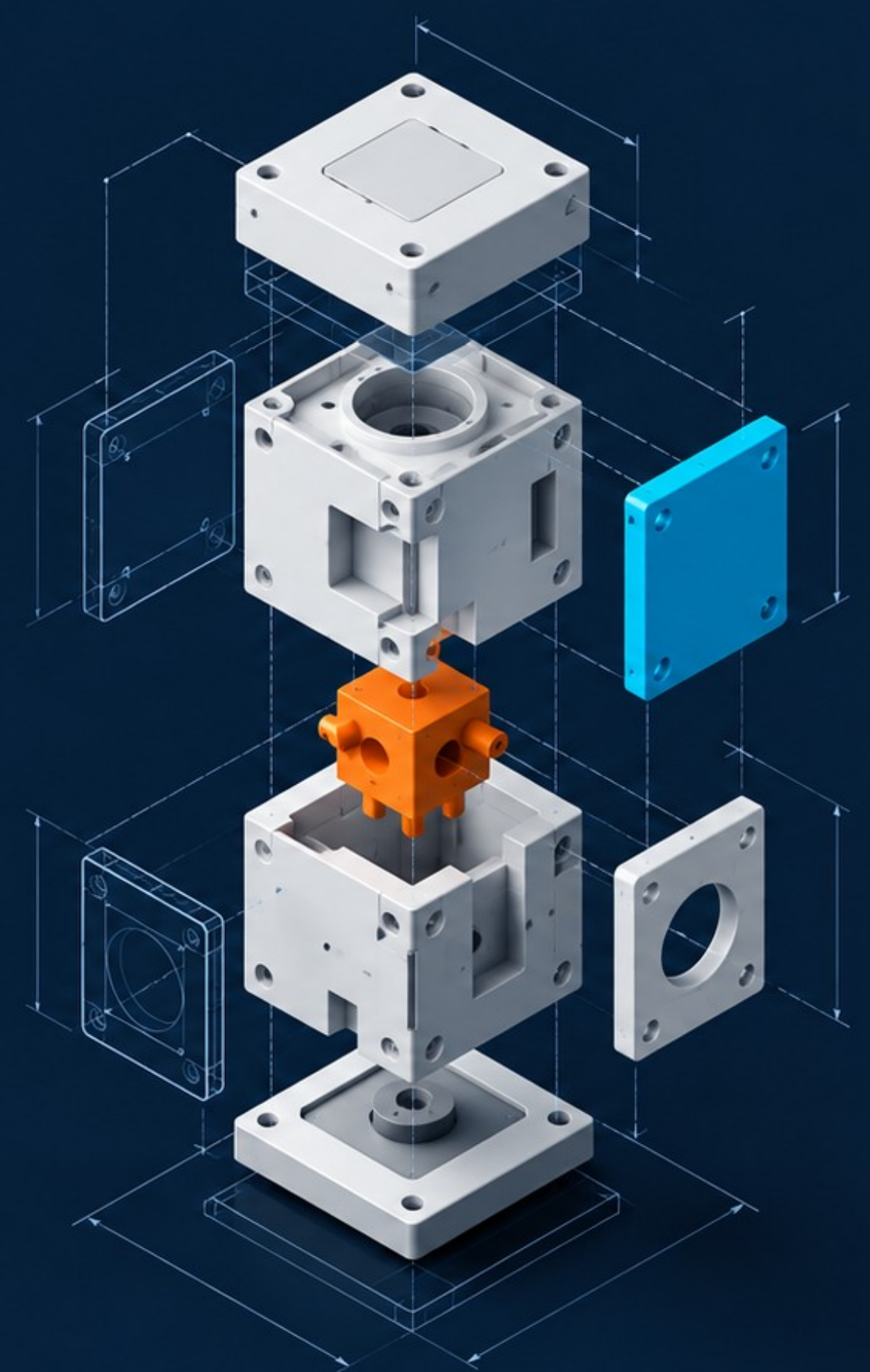
Crea extensiones de Business Central
con flujos de trabajo inteligentes

COAUTORES Y PONENTES

Roberto Corella

Javier Armesto

JORNADA PRÁCTICA · COMPANIAL · 1 DE OCTUBRE DE 2026 · 09:00–17:00



Lo que construiremos hoy

Una extensión AL Seguimiento de clientes con fechas, estados y una acción de revisión.

Un proceso de trabajo Requisito, especificación, agentes especializados y decisiones humanas.

Un conjunto reutilizable Instrucciones, skill, plugin y paquete para el equipo.

La entrega incluye código y las razones para aceptarlo.

Programa de la mañana

Horario	Trabajo
09:00–09:20	Asistentes, harness y responsabilidades
09:20–10:10	Primitivas y laboratorio de reglas de fechas
10:10–10:45	Crear y utilizar un plugin
10:45–11:00	Descanso
11:00–11:30	Agent Debug Logs · Coach opcional
11:30–12:30	ALDC: arquitectura, especificación y checkpoint
12:30–13:15	Implementación delegada

Programa de la tarde

Horario	Trabajo
13:15–14:15	Comida
14:15–14:40	Revisión y corrección
14:40–15:10	BCQuality: criterio, cita y evidencia
15:10–15:30	Aceptación y demo en Business Central
15:30–15:45	Descanso
15:45–16:20	APM: distribuir el contexto del equipo
16:20–16:50	Aceptación por parejas y diseño de vuestro flujo
16:50–17:00	Retrospectiva y siguientes pasos

De una petición a un proceso revisable

Conversación puntual

El desarrollador explica la tarea.

El asistente propone cambios.

La revisión depende de lo que se haya recordado.

Ingeniería de agentes

El alcance queda escrito y aprobado.

Cada rol recibe entradas y produce una entrega.

Las comprobaciones sostienen la aceptación.

Añadimos estructura cuando ayuda a controlar el cambio.

De conversar con prompts a un método

Más estructura, más control



No cambia el modelo: cambia dónde vive el conocimiento y quién decide.

Harness, ALDC y persona responsable

Harness

Entorno que ejecuta al agente: VS Code + GitHub Copilot Chat.

ALDC

Roles, contratos y flujos de desarrollo

Persona

Alcance, excepciones y aceptación

¿Qué es un harness?

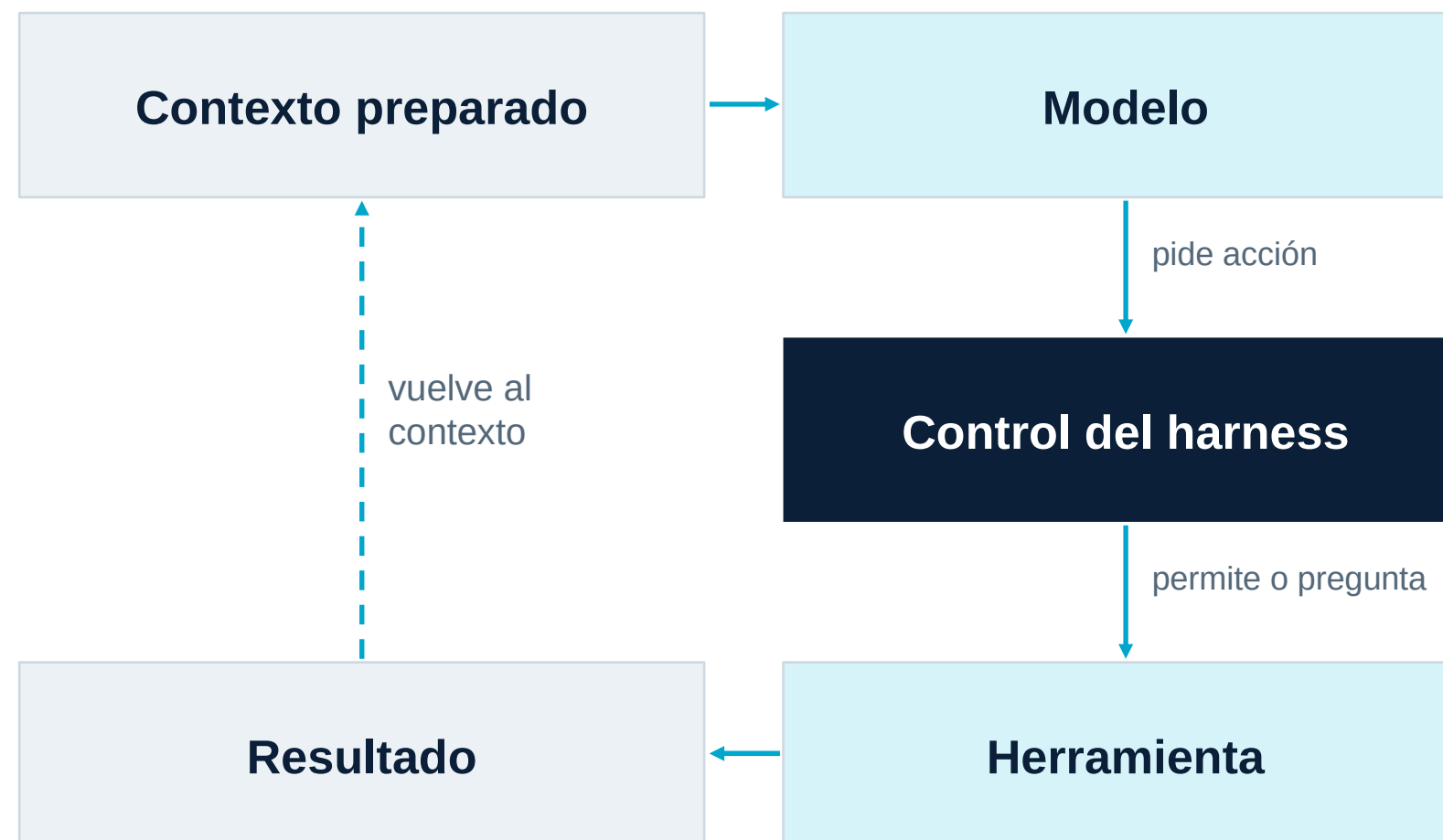
El programa que ejecuta al agente: prepara el contexto, llama al modelo, controla qué herramientas puede usar y le devuelve los resultados.

APORTA

Sesión, herramientas, permisos y registro de la ejecución.

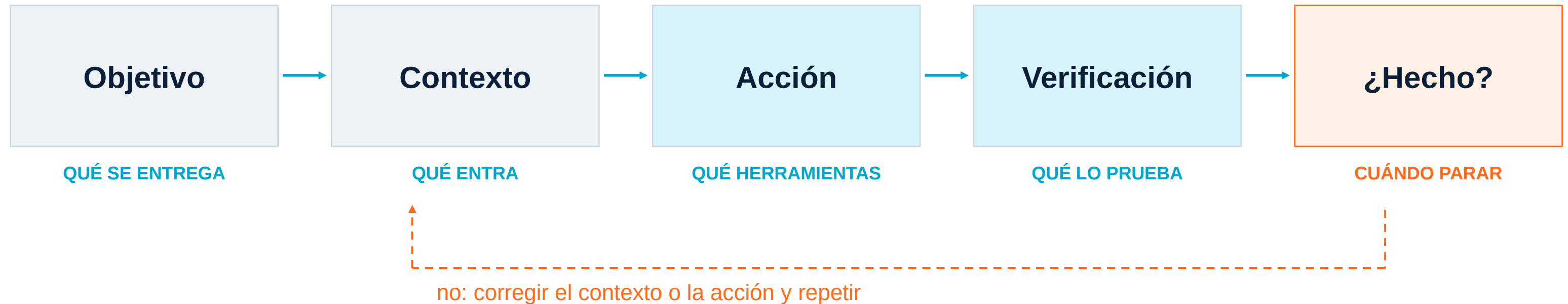
EJEMPLOS

VS Code + Copilot Chat (hoy) · Copilot CLI · Claude Code · Codex



El mismo modelo en otro harness se comporta distinto: cambian contexto, herramientas y permisos.

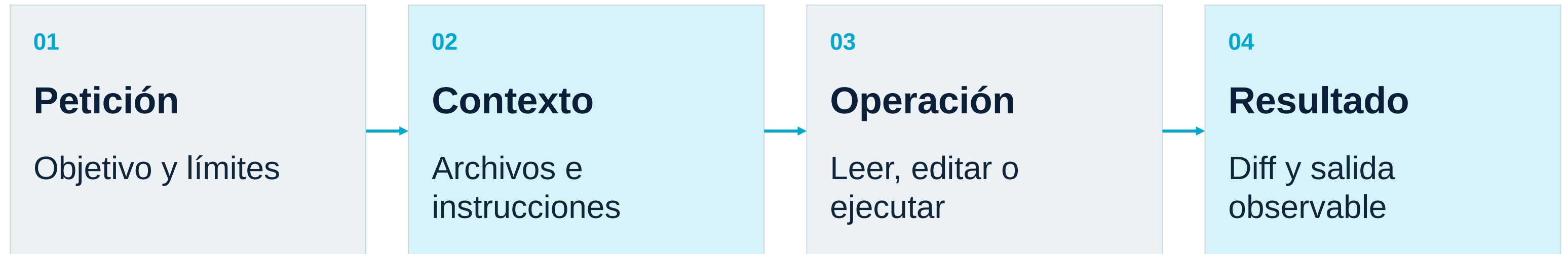
Loop engineering: diseñar el bucle



El prompt es solo una parte. Diseñamos qué contexto se carga, qué herramientas hay, qué evidencia cuenta como prueba, cuándo se detiene el bucle y dónde decide una persona.

Un buen bucle falla pronto, deja evidencia y sabe cuándo parar.

Una ejecución con herramientas



Para entender una ejecución, miramos qué contexto y qué operaciones utilizó.

Permisos y decisiones humanas



Punto de partida del participante

Workspace propio	Abrir el starter del ejercicio, con App y Test separados.
Asistente disponible	GitHub Copilot Chat en VS Code (modo agente), ALDC y APM instalados.
Entorno Business Central	Símbolos, sandbox y usuario preparados; apm --version responde.
Trabajo en parejas	Una persona conduce; la otra cuestiona y revisa.

Preparar los laboratorios · Tu repositorio



Cierre: commit «preparado para la jornada» · la guía completa está en docs/preflight.md

La instalación se hace antes: en la sala solo comprobamos.

Preparar los laboratorios · Prueba breve

☐	Contrato	El chat lee contract.es.md y resume entradas, salidas y decisiones abiertas.
☐	Símbolos y Learn	Consulta un símbolo real de Customer y una página de Microsoft Learn.
☐	Tests del starter	Publica App y Test: fallan exactamente C02, C03, C04, C11 y C12.
☐	Agentes ALDC	Aparecen Architect, AL Spec Agent, Conductor y Developer Reviewer.
☐	APM y registro	apm --version responde; Agent Debug Logs se abre desde el chat.

Si una comprobación falla, anota operación, versión y mensaje: no la des por resuelta.

Cómo trabajamos en cada laboratorio

01 Guía labs/jornada/0N-....md: pasos y prompts copiables. La slide muestra solo el núcleo.	02 Sesión Chat nuevo por lab, con el modo y el agente que indica la guía.	03 Pareja Una persona conduce; la otra cuestiona y revisa. Cambiad en cada lab.	04 Evidencia evidence/labNN-*.md. git add por ruta y commit al cerrar el lab.
--	--	--	--

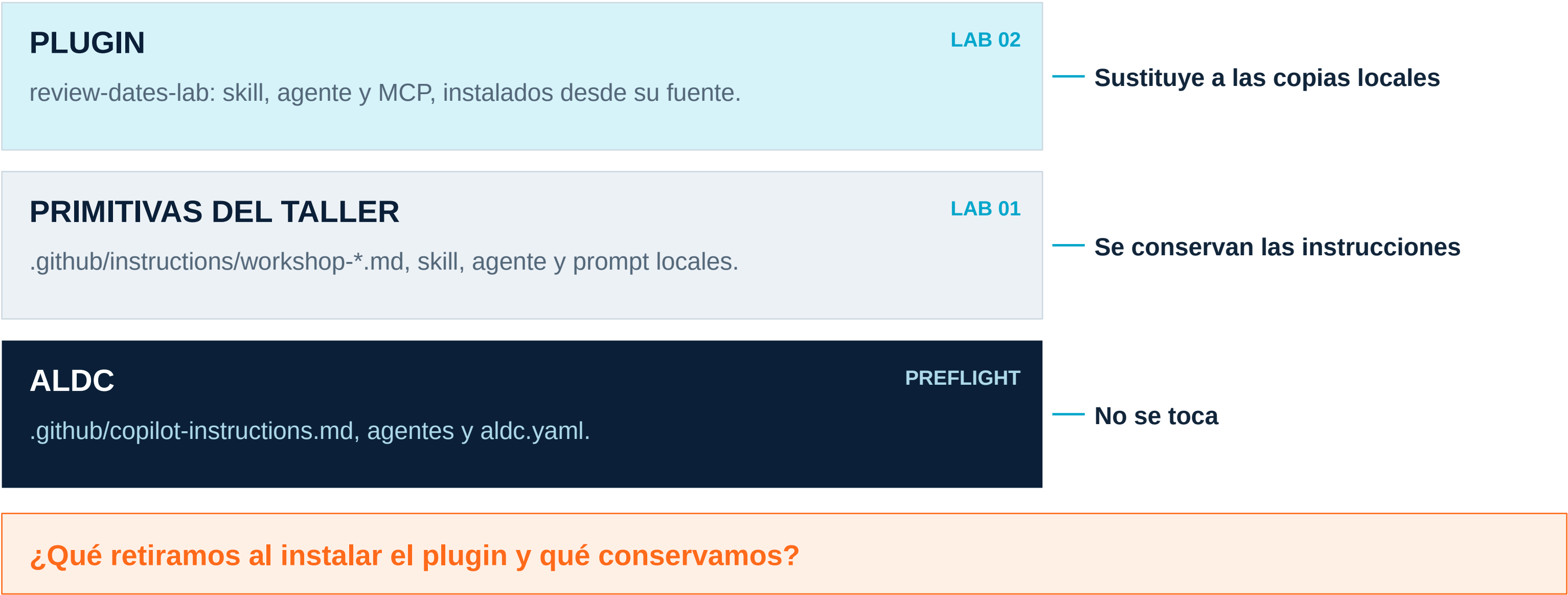
ASÍ LO VERÁS EN LAS SLIDES

TÚ lo haces tú	Agente lo hace el agente	DECIDE UNA PERSONA aprueba o devuelve una persona
---------------------------	-------------------------------------	--

Si os atascáis: el instructor tiene checkpoints preparados; no rehagáis desde cero.
Cabecera de sección en naranja: la slide es trabajo vuestro (ficha de lab o puesta en común).

Cada lab deja algo que otra persona puede revisar.

Tres capas de contexto



¿Qué es una primitiva?

La unidad mínima de contexto o capacidad que el harness sabe encontrar y cargar: un archivo con formato conocido, una ubicación y un momento de activación.

Formato	Ubicación	Activación
Markdown con frontmatter, o JSON para MCP	.github/ del proyecto, perfil de usuario o plugin	Siempre, por patrón, por descripción o al invocarla

CUÁNDO ENTRA EN EL CONTEXTO



← siempre cargado

bajo demanda →

Colocar una regla es decidir cuándo entra en el contexto y cuánto cuesta.

Las primitivas del sistema

Primitiva	Responsabilidad	Ejemplo del workshop
Instrucción	Regla estable o de alcance definido	Conservar firmas e IDs
Prompt	Petición repetible	Revisar el cambio actual
Skill	Procedimiento reutilizable	Comprobar reglas de fechas
Agente	Rol y capacidades	Revisor del incremento
Herramienta / MCP	Consulta u operación	Buscar símbolos o ejecutar tests

Primitiva · Instrucciones

CONSULTA · REPASO RÁPIDO

QUÉ ES

Reglas estables del proyecto o de un tipo de archivo.

SE CARGA

Siempre (copilot-instructions.md) o cuando el archivo casa con applyTo.

ÚSALA PARA

Convenciones, límites y lo que no se debe adivinar.

EVITA

Procedimientos largos: eso es una skill.

```
.github/instructions/workshop-al.instructions.md
```

```
---  
description: Scope and evidence conventions  
applyTo: "**/*.al"  
---  
Use the approved increment  
specification as the source of  
business behavior. Keep the  
starter's version, runtime and  
object range.
```

Primitiva · Prompts

CONSULTA · REPASO RÁPIDO

QUÉ ES

Una petición repetible con nombre.

SE CARGA

Cuando alguien la invoca con /review-followup.

ÚSALA PARA

Tareas que el equipo pide igual cada vez.

EVITA

Reglas permanentes: no se aplican si nadie lo invoca.

[.github/prompts/review-followup.prompt.md](#)

```
---  
name: review-followup  
description: Contrastar el estado de  
             revisión con el contrato  
agent: ask  
---  
Lee contract.md y el diff.  
Comprueba fecha vacía, anterior,  
igual y posterior...
```

Primitiva · Skills

CONSULTA · REPASO RÁPIDO

QUÉ ES

Un procedimiento con pasos, criterios y recursos en su carpeta.

SE CARGA

Cuando la tarea encaja con su descripción, o si se pide explícitamente.

ÚSALA PARA

Saber hacer reutilizable: revisar, migrar, probar.

EVITA

Descripciones vagas: el agente no la elegirá.

[.github/skills/review-date-rules/SKILL.md](#)

```
---
name: review-date-rules
description: Revisar reglas AL de
  fechas de seguimiento... Usar al
  comprobar el estado de revisión
---
Leer el contrato vigente...
Consultar boundary-cases.md
```

Primitiva · Agentes

CONSULTA · REPASO RÁPIDO

QUÉ ES

Un rol: instrucciones, herramientas permitidas y, si hace falta, modelo.

SE CARGA

Al elegirlo en el chat o cuando otro agente le delega.

ÚSALA PARA

Separar responsabilidades: diseñar, implementar, revisar.

EVITA

Limitar solo con texto: quita la herramienta de edición.

[.github/agents/followup-reviewer.agent.md](#)

```
name: Follow-up Reviewer
description: Revisar el contrato
           y cambios de Customer Follow-up
tools: ['search/codebase',
        'search/usages']
```

Revisa el cambio AL frente al
contrato. No edites.

Primitiva · Herramientas y MCP

CONSULTA · REPASO RÁPIDO

QUÉ ES

Operaciones que el agente puede pedir: consultar o actuar.

SE CARGA

Cuando el agente decide llamarla y el harness la permite.

ÚSALA PARA

Documentación, símbolos AL, compilar, ejecutar tests.

EVITA

Acumular servidores: cada herramienta amplía permisos.

mcp.json

```
{
  "mcpServers": {
    "microsoft-learn": {
      "type": "streamable-http",
      "url": "https://learn.microsoft.com/api/mcp"
    }
  }
}
```

Dónde colocar cada regla

Necesidad	Lugar propuesto	Motivo
Convención de todo el proyecto	Instrucciones del proyecto	Se aplica de forma transversal
Patrón exclusivo de archivos AL	Instrucción con applyTo	Acota el contexto por archivo
Comprobar fechas y persistencia	Skill de revisión	Reúne un procedimiento y sus recursos
Revisar sin implementar	Agente revisor	Separa responsabilidad y herramientas
Repetir la revisión del diff	Prompt reutilizable	Evita reescribir la petición

Instrucciones por capas

```
.github/  
  copilot-instructions.md      ← ALDC · no tocar  
  instructions/  
    workshop-project.instructions.md  
    workshop-al.instructions.md  
  agents/  
    followup-reviewer.agent.md  
  skills/  
    review-date-rules/SKILL.md
```

ALDC

copilot-instructions.md lo instala el preflight.

Taller

workshop-project y workshop-al (*.al).

Rol y tarea

Responsabilidad y procedimiento.

Tras el Lab 01 · ALDC del preflight + primitivas del taller

Una instrucción AL con alcance explícito

```
---  
applyTo: "**/* .al"  
---
```

Conserva las firmas y los IDs suministrados.
Recibe la fecha de referencia como parámetro.
No alteres campos ajenos a la revisión.
Documenta qué comprobaciones has ejecutado.

Reglas observables

Podemos revisar su cumplimiento en el diff.

Alcance limitado

No convertir una instrucción AL en una guía universal.

Una skill reúne un procedimiento

```
---  
name: review-date-rules  
description: Revisa estados y fechas de seguimiento.  
---  
  
1. Lee el contrato y la fecha de referencia.  
2. Comprueba vacío, pasado, hoy y futuro.  
3. Revisa reprogramación y repetición.  
4. Comprueba persistencia y otros campos.  
5. Relaciona cada resultado con un caso.
```

Activación

El nombre y la descripción ayudan a localizarla.

Recursos

Los casos detallados pueden vivir en otro archivo.

SKILL.md · extracto simplificado

El contrato del agente revisor

Recibe

Especificación aprobada.

Diff y archivos relevantes.

Resultados disponibles de compilación y tests.

Devuelve

Hallazgo o comprobación concreta.

Archivo, criterio y evidencia.

Corrección propuesta o ausencia de hallazgos.

Un rol es útil cuando su responsabilidad y su entrega son distintas.

El caso: seguimiento de clientes

REQUISITO / OBSERVACIÓN

“El equipo comercial necesita saber qué clientes debe revisar y dejar preparada la siguiente revisión.”

Pantalla

Customer Card

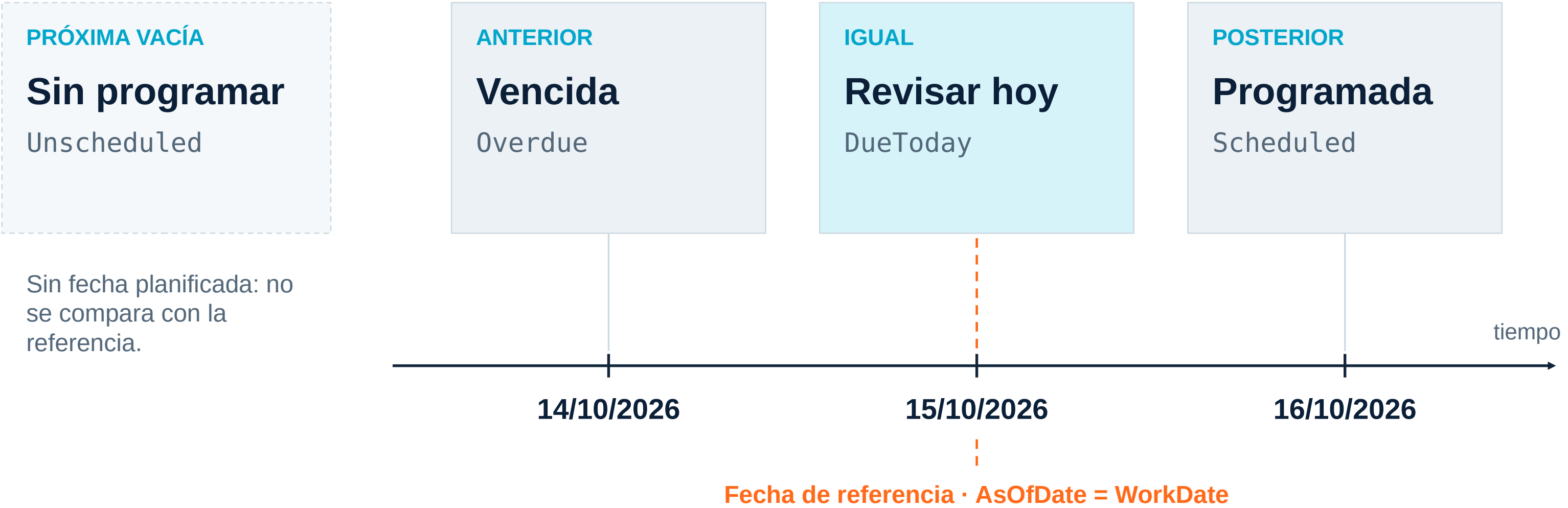
Información

Última fecha, próxima fecha y estado

Acción

Mark as reviewed

Cuatro estados; una fecha de referencia



WorkDate llega desde la interfaz

```
GetReviewStatus(  
    NextReviewDate: Date;  
    AsOfDate: Date)  
  
// La interfaz utiliza WorkDate().  
// Los tests proporcionan una fecha fija.  
// La función calcula; no modifica Customer.
```

Separación

La interfaz aporta la fecha de la sesión.

Reproducibilidad

El mismo dato y referencia producen el mismo estado.

Contrato de la operación · firma resumida

Lab 01 · Contrato y contexto

Objetivo: Completar GetReviewStatus usando reglas explícitas.

20

MINUTOS APROX.

- 01 Revisad el contrato y las instrucciones aplicables.
- 02 Completad la función y revisad el diff.
- 03 Ejecutad C01–C06 y después C01–C12: esperado 10/12.

PUNTO DE PARTIDA

Starter, contrato y skill de fechas.

CARPETA DEL REPOSITORIO

labs/jornada/
01-contrato-y-contexto.md

ENTREGA

Diff acotado + tabla de casos observados.

Lab 01 · Petición inicial

PROMPT DE TRABAJO

Lee el contrato de Customer Follow-up y las instrucciones aplicables. Completa únicamente GetReviewStatus. Mantén la fecha de referencia explícita y las validaciones existentes. Usa review-date-rules para revisar los límites. Devuelve el diff y distingue resultados ejecutados de comprobaciones pendientes.

Chat nuevo · modo Agent · Follow-up Reviewer solo revisa. Texto copiable en las notas.

Resultados esperados de C01–C06

Caso	Comprobación	Esperado
C01	Próxima fecha vacía	Sin programar
C02	Próxima fecha anterior	Vencida
C03	Misma fecha	Revisar hoy
C04	Próxima fecha posterior	Programada
C05	Referencia vacía	Error de validación
C06	Próxima fecha de cierre	Error de validación

Suite completa: 10/12. C11 y C12 esperan a MarkReviewed (Lab 05).

Puesta en común · ¿Qué regla ayudó?

01 ¿Qué decisión habría quedado implícita sin el contrato?

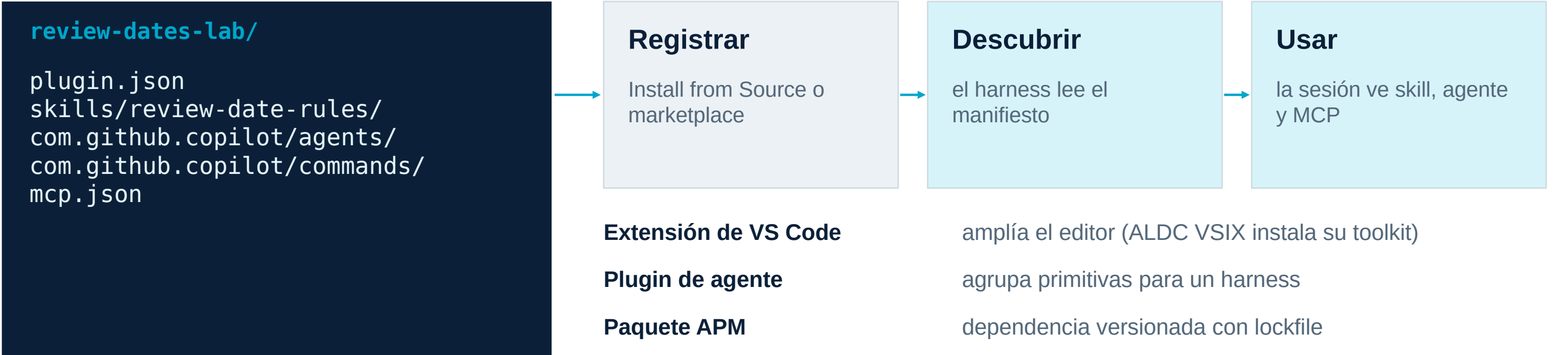
02 ¿Qué instrucción o skill influyó en el cambio?

03 ¿Qué prueba os obligó a corregir una suposición?

Conservad esta implementación para el ciclo con ALDC.

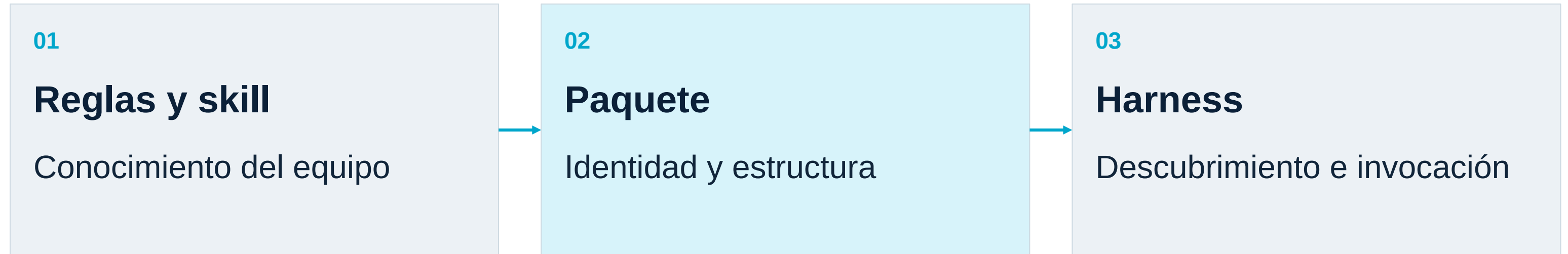
¿Qué es un plugin?

Un paquete que agrupa primitivas (skills, agentes, comandos y servidores MCP) con un manifiesto, para que un harness las instale y las descubra juntas.



Un plugin hace portable el contexto; no garantiza que el agente lo use.

De archivos sueltos a un plugin



Estructura del plugin del ejercicio

```
review-dates-lab/  
  plugin.json  
  skills/review-date-rules/  
    SKILL.md  
    boundary-cases.md  
mcp.json  
com.github.copilot/  
  agents/  
  commands/
```

Compartido

Skill y configuración MCP.

Específico de Copilot

Agentes y comandos en su espacio de nombres.

Identidad y registro local

```
{
  "$schema":
    "https://agent-plugins.org/schemas/1.0.0/plugin.schema.json",
  "name": "review-dates-lab",
  "version": "0.2.0"
}

// Alternativa en settings.json
"chat.pluginLocations": {
  "/ruta/absoluta/review-dates-lab": true
}
```

plugin.json · registro alternativo en settings.json

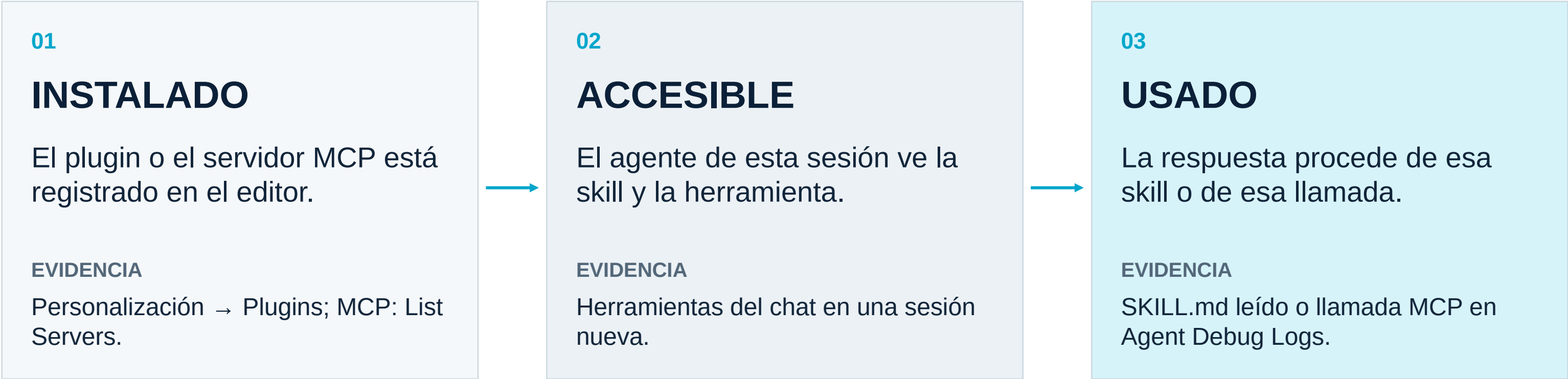
Manifiesto

Describe el paquete.

Registro

Personalización → Plugins →
Install from Source.

Instalado, accesible, usado



Learn MCP aporta documentación; los símbolos, objetos disponibles; compilación y tests, resultados. Cada fuente deja una evidencia distinta.

Una respuesta convincente no prueba su origen: nombrad la evidencia de cada nivel.

Lab 02 · Plugin utilizable

Objetivo: Reutilizar la revisión de fechas en una sesión nueva.

20

MINUTOS APROX.

- 01 Preparad manifiesto, skill, agente y MCP desde las plantillas.
- 02 Retirad duplicados, instalad el plugin y abrid una sesión nueva.
- 03 Invocad la revisión y localizad una consulta con su fuente.

PUNTO DE PARTIDA

Plantillas de plugin y función de estado.

CARPETA DEL REPOSITORIO

labs/jornada/
02-plugin-utilizable.md

ENTREGA

Ruta del plugin +
componente usado +
resultado de revisión.

Lab 02 · Pasos para montar el plugin

Copiar y completar	Crear una carpeta propia desde templates/plugin.
Registrar	Retirar duplicados locales e instalar con Install from Source.
Abrir una sesión	Localizar skill, agente y herramientas disponibles.
Utilizar	Revisar la función y consultar una API AL en Learn.

Cuando el plugin no aparece

Síntoma	Primera comprobación
No se descubre	Ruta real, manifiesto y sufijos de archivo
Aparece una skill distinta	Copias duplicadas en workspace o usuario
El agente no usa la skill	Petición explícita y contexto de activación
No se ve una herramienta MCP	Servidor disponible y herramientas habilitadas

Puesta en común · Del paquete a la sesión

- 01 ¿Qué componente se ha cargado desde el plugin?
 - 02 ¿Qué operación y resultado podéis enseñar?
 - 03 ¿De dónde vino el MCP: del plugin o de otra configuración?
-

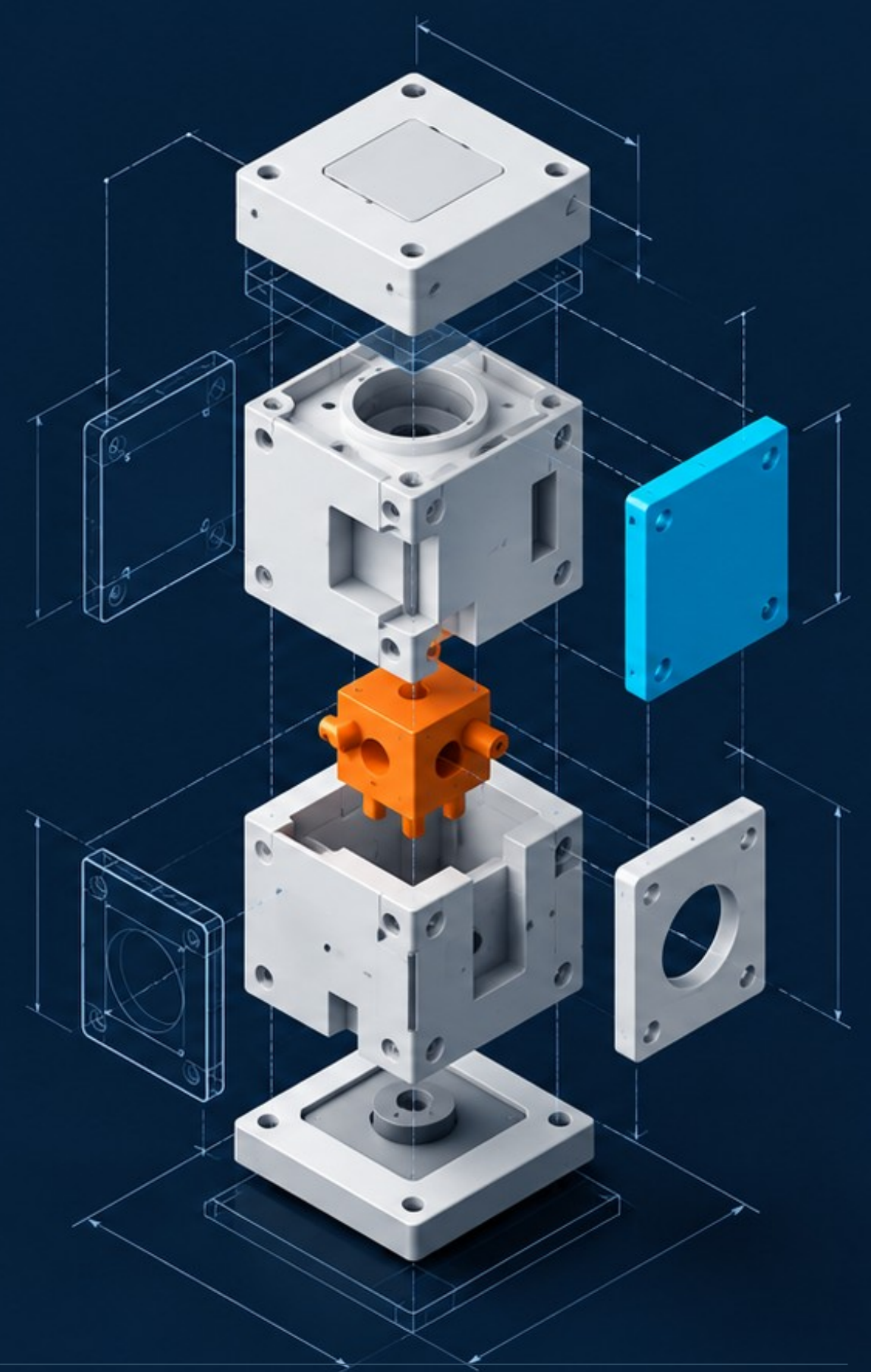
El procedimiento ya se puede reutilizar fuera del primer workspace.

PAUSA

Descanso

Volvemos a las 11:00

Guardad vuestro trabajo antes de salir.



Una respuesta no describe toda la ejecución

REQUISITO / OBSERVACIÓN

“He revisado los casos límite.”

Pregunta

¿Qué contrato y qué archivos leyó?

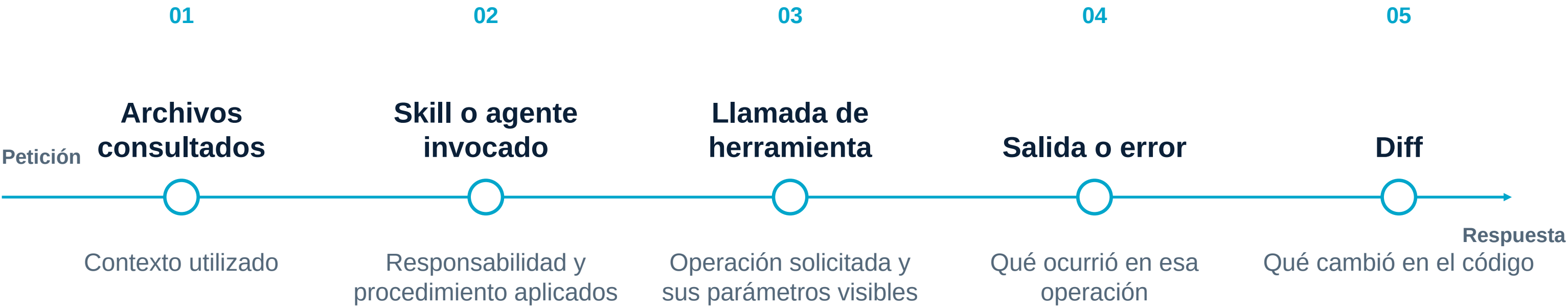
Operación

¿Qué herramienta invocó?

Evidencia

¿Qué resultado podemos contrastar?

Qué mirar en una sesión



Cada señal es un punto de la traza que se puede contrastar.

Apoyo opcional: AI Engineer Coach

Vista	Pregunta para el desarrollador
Context Health	¿Qué instrucciones se solapan o necesitan aclaración?
Anti-Patterns	¿Qué patrón de uso conviene revisar?
Skill Finder	¿Qué tarea repetida merece una skill?

El Lab 03 se completa con Agent Debug Logs. Coach aporta señales adicionales; una puntuación no prueba que el código sea correcto.

Demo · De una señal a una mejora

Localizar	Elegir una sesión del laboratorio y una señal concreta.
Contrastar	Abrir la instrucción, petición u operación relacionada.
Modificar	Cambiar una sola regla o parte de la petición.
Repetir	Rehacer la misma tarea y comparar el comportamiento.

Lab 03 · Una mejora observable

Objetivo: Mejorar una ejecución a partir de lo que ha ocurrido.

15

MINUTOS APROX.

- 01 Localizad una señal en Agent Debug Logs (Coach, opcional).
- 02 Cambiad una instrucción en la skill de vuestra copia del plugin.
- 03 Repetid la tarea y comparad operaciones y salida.

PUNTO DE PARTIDA

Sesión del plugin y registros disponibles.

CARPETA DEL REPOSITORIO

labs/jornada/
03-mejora-observable.md

ENTREGA

Señal inicial + cambio + observación posterior.

Puesta en común · Intervenciones útiles

01 ¿Qué problema habéis observado, exactamente?

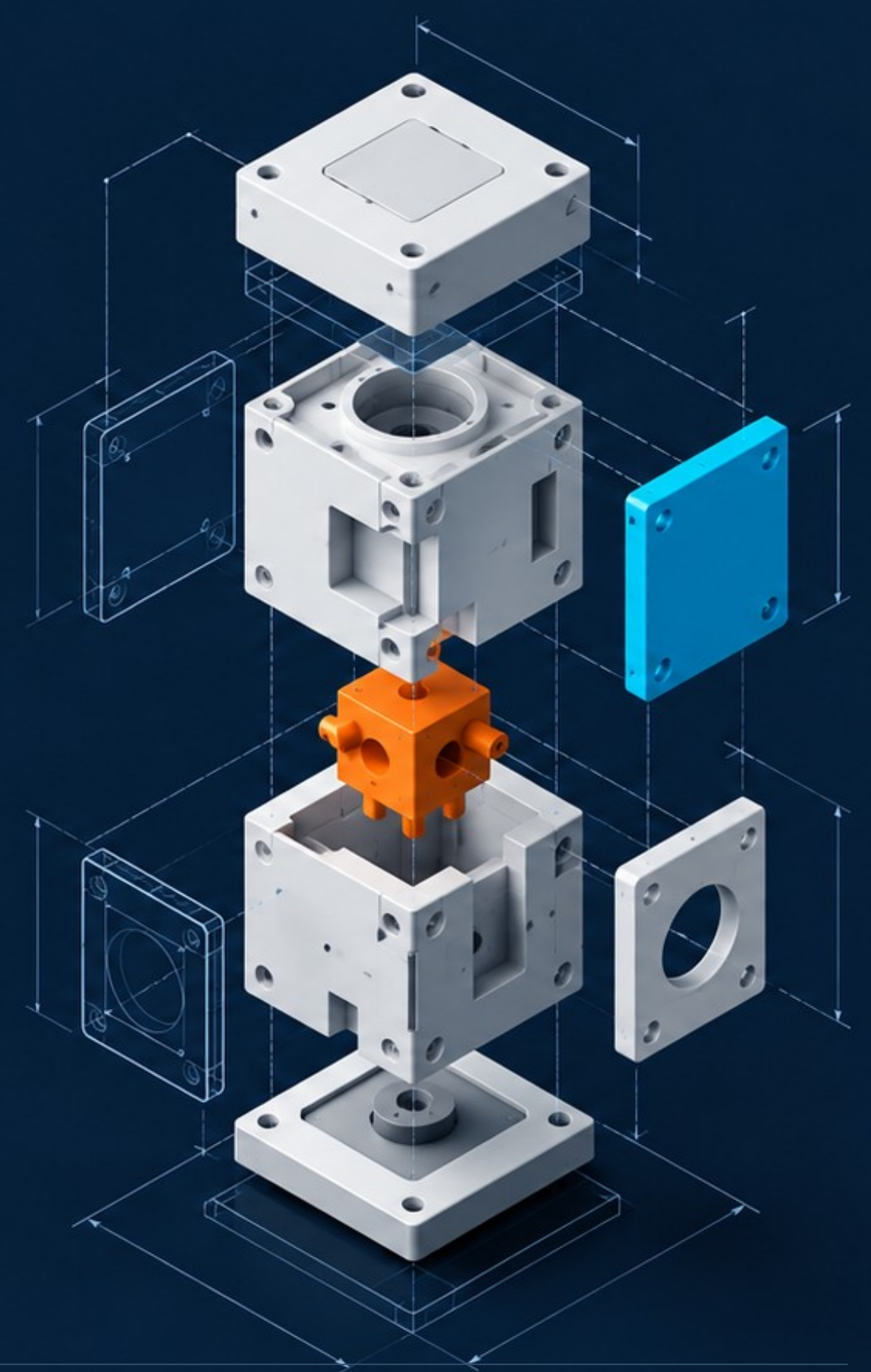
02 ¿Qué cambiasteis y por qué?

03 ¿Qué evidencia permite decir que mejoró?

Una ejecución muestra un efecto, no una regla. Sin defecto inicial, se registra así.

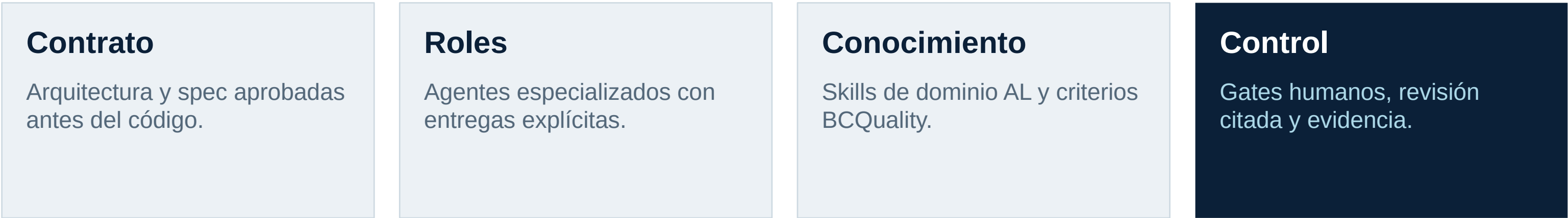
ALDC: un método de desarrollo con agentes

ALDC ya está instalado desde el preflight. Ahora usamos su flujo: roles, entregas y aprobaciones.



¿Qué es ALDC?

AL Development Collection: agentes, skills, instrucciones y flujos para desarrollar extensiones de Business Central con contrato, fases y aprobación humana.



DISTRIBUCIONES



En el taller: ALDC 5.0.0 como extensión de VS Code, con GitHub Copilot Chat.

Agentes principales de ALDC

HOY Architect Diseño, alcance y descomposición en specs.	HOY Spec Agent De arquitectura aprobada a contrato de especificación.	HOY Conductor Planifica, delega y revisa por fases con gates.	Developer Implementación directa de un cambio acotado.
HOY Developer Reviewer Revisión independiente antes de la aprobación.	Dredd Auditoría independiente y consultiva; no edita AL.	Triage Diagnóstico de errores y su radio de afectación.	Pre-Sales Estimación y descubrimiento.

Dentro del Conductor: Planning, Implementation y Review subagents

Cada agente recibe una entrada y produce una entrega que otra persona puede revisar.

Skills principales de ALDC

BASE

skill-api

skill-events

skill-permissions

skill-performance

skill-testing

skill-debug

skill-copilot

RECOMENDADAS

skill-pages

skill-migrate

skill-translate

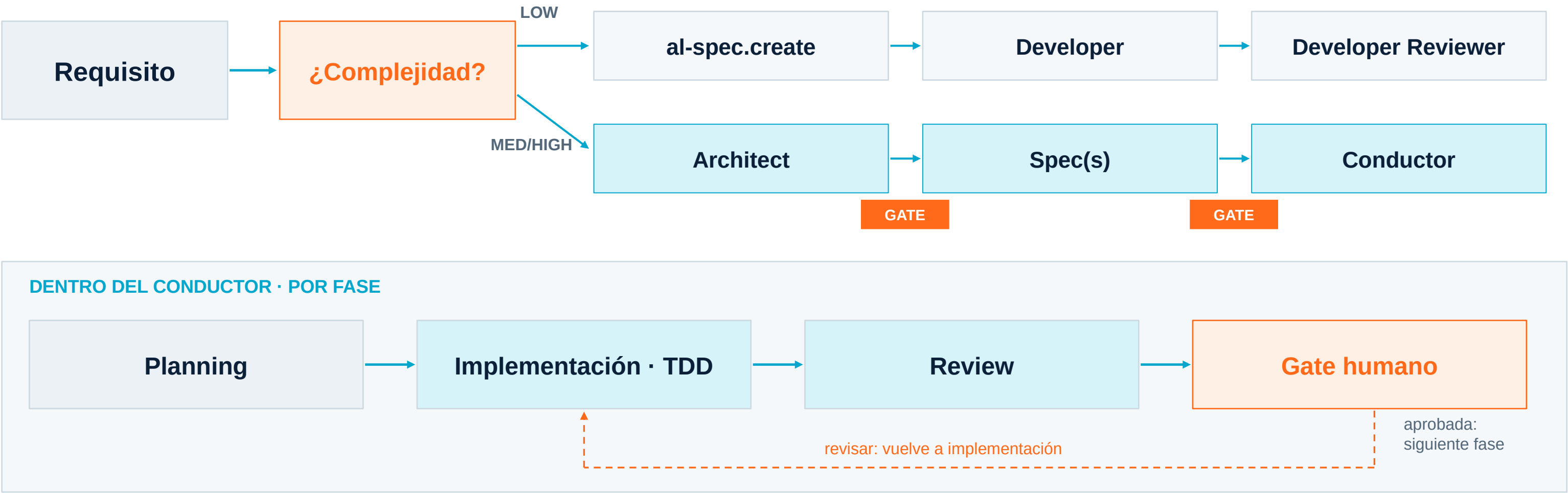
skill-estimation

REVISIÓN COMPARTIDA

skill-al-review-pipeline

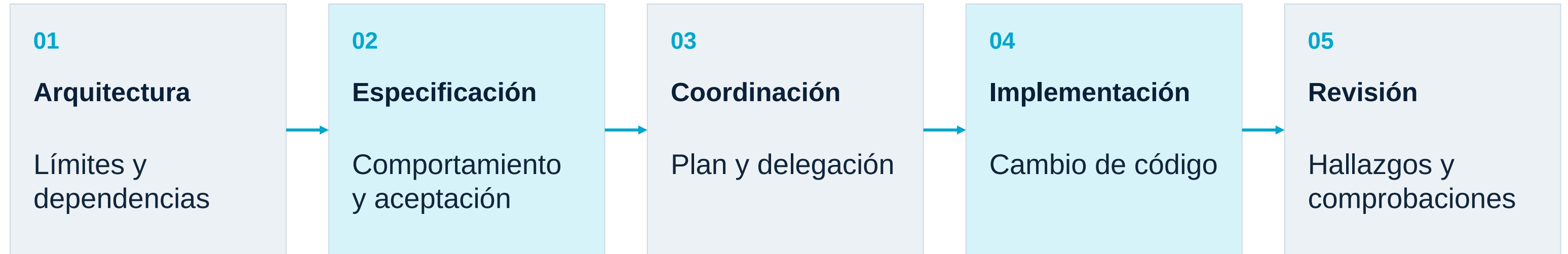
Una skill cargada no es una skill aplicada: pedid que el agente diga cuál usó y para qué.

El loop de ALDC

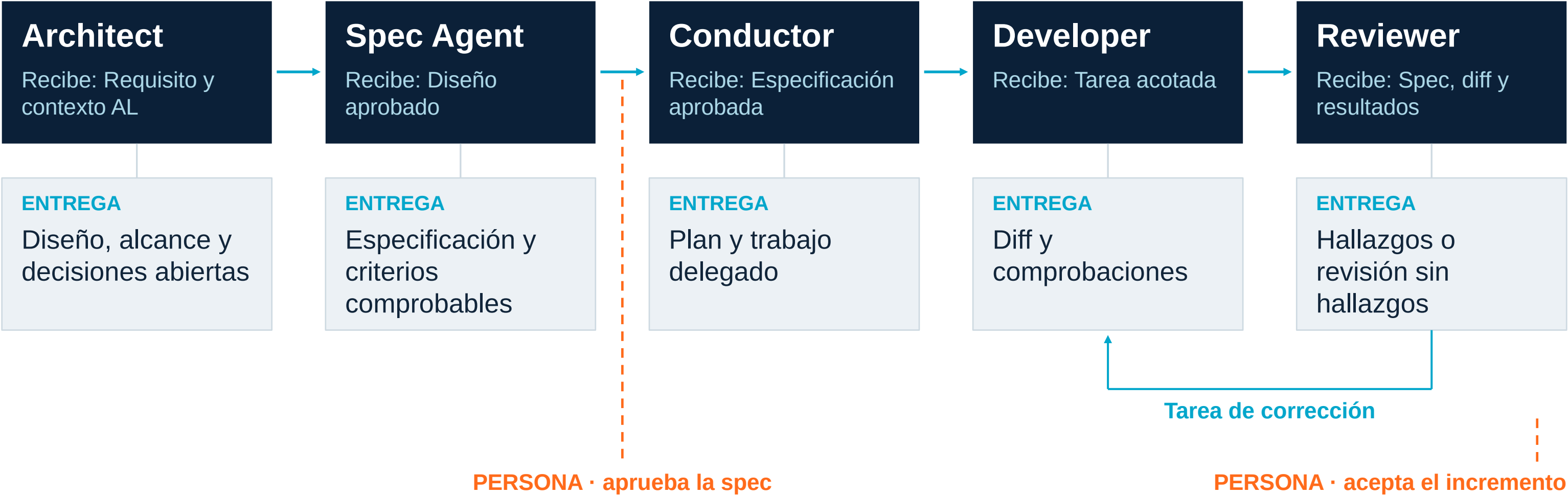


Varias specs con dependencias forman un grafo; cada fase es un bucle con salida humana.

El recorrido del incremento



Roles y entregas



Cuánta coordinación necesita el cambio

Cambio pequeño y localizado

Una regla clara.

Pocos archivos afectados.

Spec, implementación y revisión pueden bastar.

Cambio con varias dependencias

Decisiones de diseño abiertas.

Trabajo que se puede separar.

Coordinación y entregas explícitas ayudan.

El número de agentes responde al trabajo. En el taller, Conductor es una elección docente.

El requisito completo de Customer Follow-up

REQUISITO / OBSERVACIÓN

“Desde la ficha de cliente, indicar la próxima revisión, ver su estado y marcar una revisión realizada. Al hacerlo, registrar la fecha de trabajo y programar la siguiente a 30 días naturales.”

Ya resuelto

Estados con referencia explícita

Por construir

Actualizar y guardar las dos fechas

Por demostrar

Repetición, persistencia y uso desde la ficha

El límite del incremento

Incluido

Campos de última y próxima revisión.

Estado calculado y acción en Customer Card.

Reprogramación, repetición y persistencia.

Fuera de alcance

Historial de revisiones.

Correos y tareas programadas.

Bloqueos y documentos de venta.

Qué aporta el starter

Pieza	Función	Trabajo de hoy
Table extension	Dos fechas sobre Customer	Suministrada
Page extension	Campos, estado y acción	Suministrada
Enum	Cuatro estados	Suministrado
Codeunit	Reglas y persistencia	Completar dos TODOs
Permission set y tests	Ejecución y casos C01–C12	Utilizar y revisar

Contexto que necesita Architect

Pregunta	Evidencia que buscamos
¿Dónde se añade la funcionalidad?	Customer, Customer Card y objetos del starter
¿Qué interfaces debemos mantener?	Firmas usadas por la página y los tests
¿Qué reglas son decisiones de negocio?	Contrato y casos de aceptación
¿Qué criterios de calidad aplican?	Fuente de BCQuality y su pertinencia
¿Qué objetos existen de verdad?	Símbolo Customer consultado (consultar ≠ compilar)

Petición al Architect

PROMPT DE TRABAJO

Analiza Customer Follow-up con el contrato y el starter. Define alcance, objetos, dependencias y separación entre interfaz y lógica. Consulta los símbolos y el conocimiento BCQuality disponible que resulte pertinente. Expón decisiones abiertas y criterios de diseño con su origen. Entrega arquitectura; todavía no implementes.

El texto está disponible para copiar en las notas de esta diapositiva.

De la arquitectura a la especificación

Comportamiento Entrada, salida y efectos sobre los datos.

Límites Fechas vacías, igualdad, repetición y persistencia.

Aceptación Casos verificables y criterios de calidad aplicables.

Petición al Spec Agent

PROMPT DE TRABAJO

A partir de la arquitectura aprobada, especifica GetReviewStatus y MarkReviewed. Incluye entradas, validaciones, efectos, casos C01–C12 y límites del alcance. Declara los criterios BCQuality seleccionados y sus referencias en .bcq-criteria.json. Separa las decisiones pendientes. No generes todavía el código AL.

El texto está disponible para copiar en las notas de esta diapositiva.

Reglas que deben quedar cerradas

Decisión	Contrato del ejercicio
Fecha vacía	Próxima vacía: Sin programar; referencia vacía: error
Referencia	La interfaz pasa WorkDate; la lógica recibe Date
Próxima revisión	Fecha de revisión + 30 días naturales
Repetición	Misma fecha de revisión, mismas fechas guardadas
Persistencia	Al reabrir, los valores siguen guardados
Otros campos	Se conservan; no se elevan permisos
Fecha de cierre	Próxima revisión con fecha de cierre (C): error de validación

Lab 04 · Requisito a especificación

Objetivo: Obtener una especificación que otra persona pueda revisar.

20

MINUTOS APROX.

- 01 Pedid arquitectura y resolved las decisiones de diseño.
- 02 Pasad el diseño aprobado a Spec Agent.
- 03 Contrastad la spec con los casos y criterios seleccionados.

PUNTO DE PARTIDA

Contrato, starter, ALDC y conocimiento BCQuality disponible.

CARPETA DEL REPOSITORIO

labs/jornada/
04-requisito-a-especificacion.md

ENTREGA

Spec revisable + criterios citados + decisiones abiertas.

Checkpoint humano · Antes de implementar

Alcance	¿Solo construimos el incremento acordado?
Semántica	¿Están cerrados vacío, igualdad, 30 días y repetición?
Comprobación	¿Los casos y criterios permiten revisar el resultado?
Decisión	Aprobar, devolver con cambios o detener por una duda.

Revisión cruzada · Aprobar la spec

5

MINUTOS APROX.

- 01 La otra persona explica las reglas con sus palabras.
- 02 Anotad una corrección si existe ambigüedad.
- 03 Registrad la decisión sobre la especificación final.

ENTREGA

Aprobación explícita o devolución con una pregunta concreta.

Puesta en común · Qué hemos aprobado

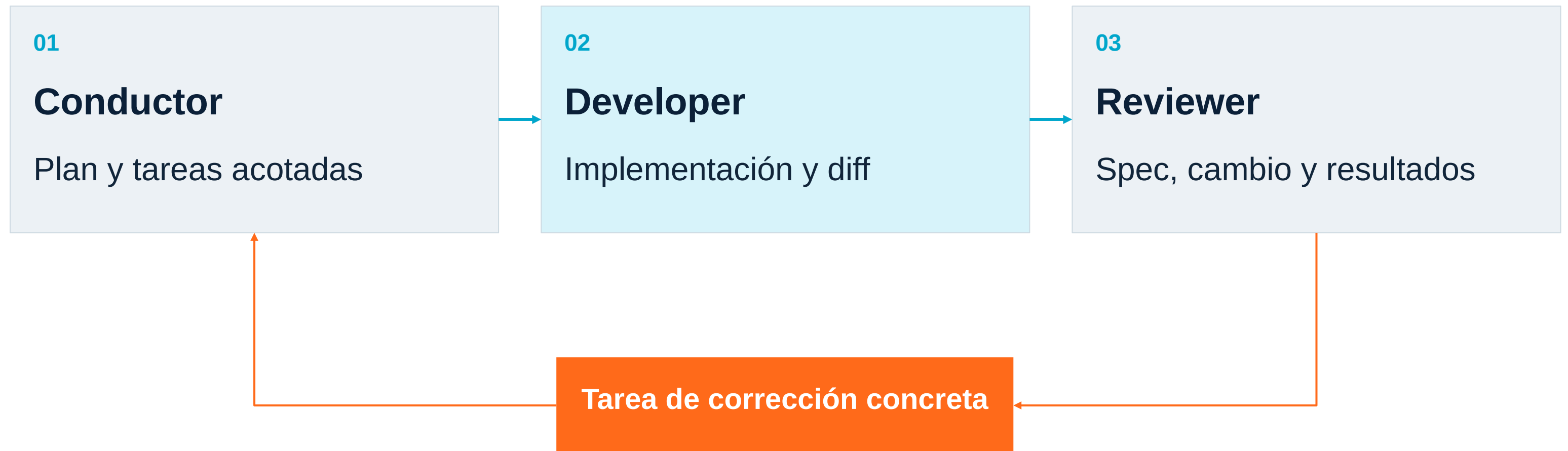
01 ¿Qué ambigüedad corrigió la revisión humana?

02 ¿Qué criterio BCQuality influyó en el diseño?

03 ¿Qué recibe exactamente el siguiente agente?

Un contrato de trabajo que ya permite implementar y revisar.

La delegación del Conductor



El cambio pendiente: MarkReviewed

```
MarkReviewed(Customer, ReviewDate)
```

```
Última revisión := ReviewDate
```

```
Próxima revisión := ReviewDate + 30 días
```

```
Guardar el cliente.
```

```
Conservar los demás campos.
```

```
Repetir con la misma fecha mantiene el resultado.
```

Origen

La revisión realizada, no la planificación anterior.

Efecto

Dos fechas persistidas en Customer.

Pseudocódigo del contrato; no es código AL para pegar

Casos que completan el incremento

Caso	Entrada o acción	Esperado
C07	Fecha de revisión vacía	Error
C08	15/10/2026 + 30 días	14/11/2026
C09	31/01/2024 + 30 días	01/03/2024
C10	15/12/2026 + 30 días	14/01/2027
C11	Revisar dos veces con la misma fecha	Mismas fechas; nombre conservado
C12	Guardar y volver a leer Customer	Fechas persistidas

Petición al Conductor

PROMPT DE TRABAJO

DECIDE UNA PERSONA

- 1 · Usa la arquitectura y la spec aprobadas. Planifica una fase mínima para MarkReviewed y la delegación. Presenta el plan y espera mi aprobación.
- 2 · «Apruebo el plan.» Procede con implementación delegada y revisión. Si necesitas publicar o ejecutar tests, indícalo y espera mis resultados.

Entre 1 y 2 decide una persona. Texto completo en la guía del Lab 05.

Lab 05 · Incremento implementado

Objetivo: Completar el comportamiento y comprobarlo con C01–C12.

25

MINUTOS APROX.

- 01 Revisad el plan y aprobadlo antes de que delegue.
- 02 Completad la persistencia y revisad el alcance del diff.
- 03 Publicad App y ejecutad C01–C12 en una ejecución nueva.

PUNTO DE PARTIDA

Arquitectura y spec aprobadas; starter con el trabajo anterior.

CARPETA DEL REPOSITORIO

labs/jornada/
05-incremento-implementado.md

ENTREGA

Cambio implementado + resultados reales + pendientes claros.

Compilar, publicar, probar, aceptar



El 10/12 era un resultado antiguo, no un motivo para cambiar la lógica.

Antes de la comida

- 01 ¿Qué parte del incremento está implementada?
 - 02 ¿Qué resultados tenéis y qué queda pendiente?
 - 03 ¿Qué hallazgo debe revisar la pareja al volver?
-

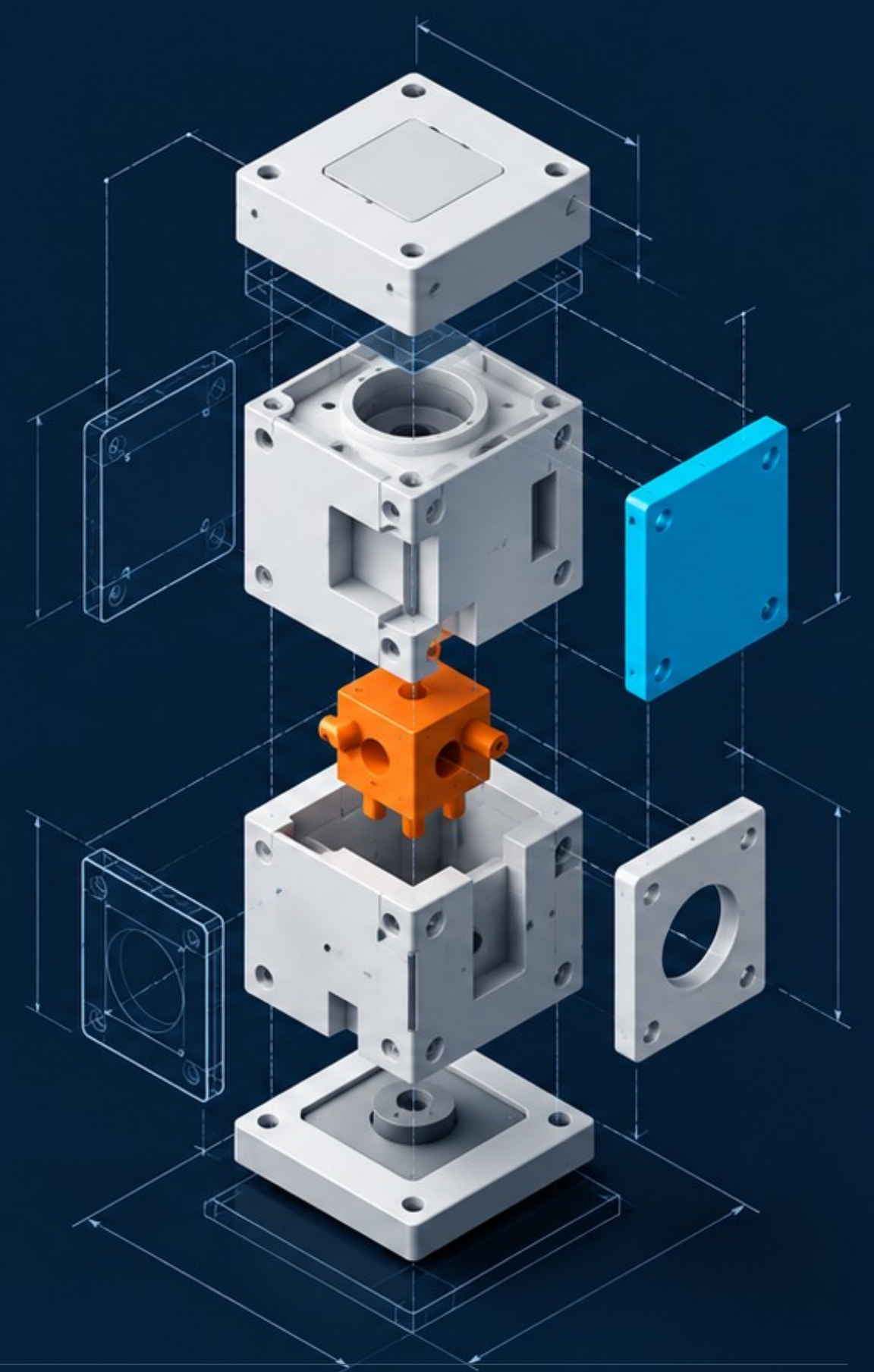
Guardad spec, diff y resultados en el workspace.

PAUSA

Comida

Volvemos a las 14:15

Guardad vuestro trabajo antes de salir.



Reanudar desde la entrega

Especificación	Qué comportamiento se aprobó.
Diff	Qué archivos y reglas han cambiado.
Resultados	Qué se ejecutó sobre ese cambio.

La revisión en ALDC

Quien implementa no se aprueba a sí mismo.



La revisión aporta criterio citado; la aceptación sigue siendo una decisión humana.

La anatomía de un hallazgo útil

Elemento	Ejemplo docente
Criterio	La misma fecha debe ser Revisar hoy
Localización	Selección de estado en GetReviewStatus
Observación	La comparación incluye la igualdad en Vencida
Consecuencia	C03 devuelve el estado incorrecto
Corrección y prueba	Separar igualdad y repetir C03 + regresión

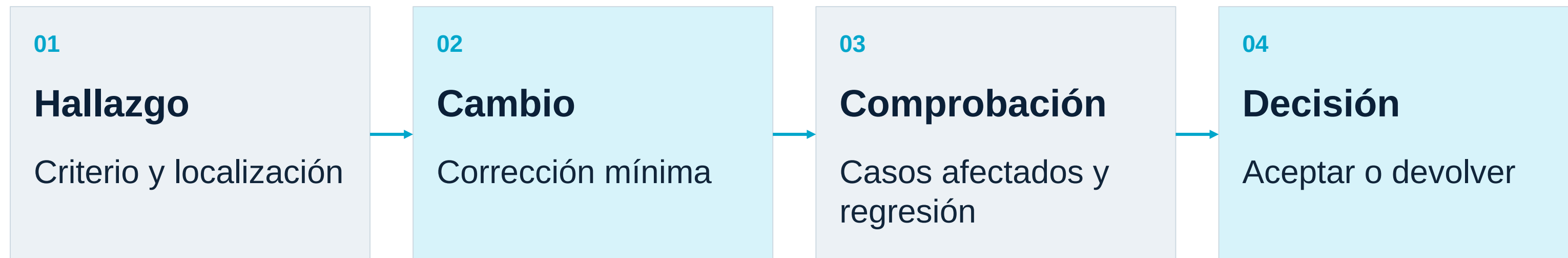
Petición de revisión

PROMPT DE TRABAJO

Revisa el diff frente a la especificación aprobada. Aplica los criterios de calidad seleccionados. Para cada hallazgo, indica archivo, comportamiento afectado, evidencia y corrección propuesta. No modifiques el código durante esta revisión. Distingue lo revisado estáticamente de lo ejecutado.

El texto está disponible para copiar en las notas de esta diapositiva.

Un ciclo de corrección acotado



Trabajo en parejas · Revisión y corrección

20

MINUTOS APROX.

- 01 Intercambiad los papeles y revisad spec, diff y resultados.
- 02 Corregid un hallazgo aplicable o documentad la revisión.
- 03 Repetid las comprobaciones necesarias sobre el cambio final.

ENTREGA

Revisión comprensible + cambio final + resultados asociados.

¿Qué es BCQuality?

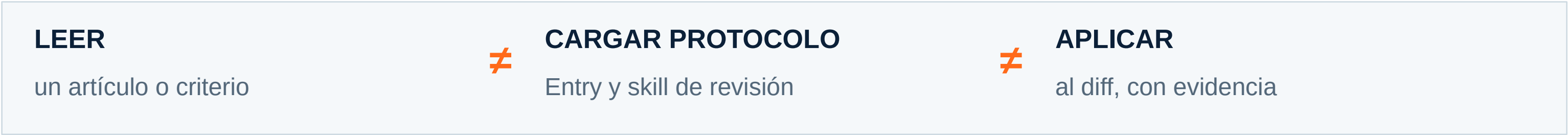
Conocimiento de calidad para AL publicado por el equipo de Business Central de Microsoft: artículos y skills de revisión que se pueden citar.



Taller: modo external-multirroot en ../bcquality, commit fijado 07e324d. Alternativa: modo plugin.

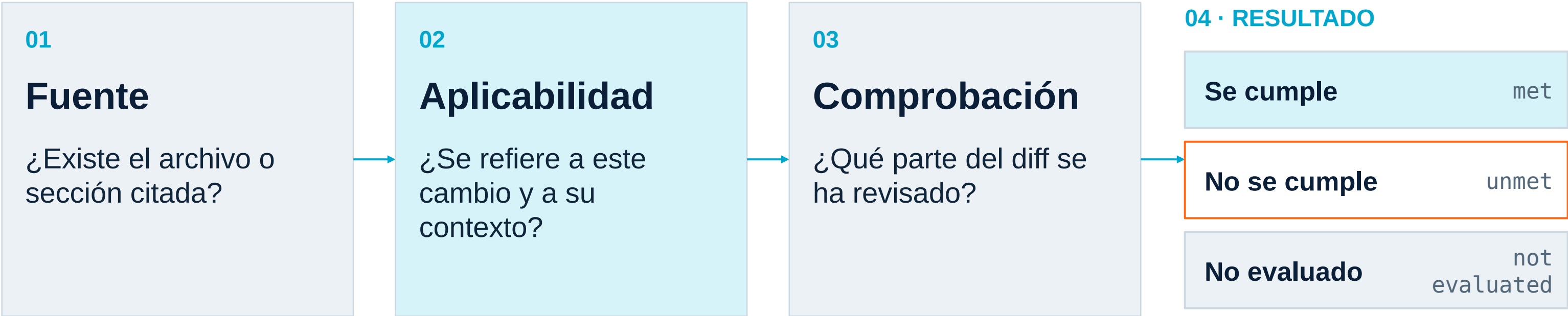
Detecta código que funciona pero no tiene calidad suficiente, antes de que aparezca en producción.

BCQuality a lo largo del desarrollo



Una skill se ejecuta siguiendo su protocolo; no necesita un servicio remoto.

Una cita tiene que explicar una decisión



«No evaluado» es un resultado válido; una ejecución inventada no lo es.

Qué demuestra cada evidencia

Evidencia	Permite evaluar	No resuelve por sí sola
Criterio citado	Regla y aplicabilidad	Comportamiento ejecutado
Revisión del diff	Implementación frente al criterio	Todas las rutas de ejecución
Compilación	Validez del código para el compilador	Reglas comerciales
Tests y recorrido	Casos ejecutados y comportamiento observado	Todo escenario posible

Lab 06 · Revisión citada

Objetivo: Seguir un criterio desde su fuente hasta la decisión.

15

MINUTOS APROX.

- 01 Abrid una referencia seleccionada en la especificación.
- 02 Contrastad su aplicación al diff y la evaluación del revisor.
- 03 Decidid y, si hay cambio, repetid la comprobación afectada.

PUNTO DE PARTIDA

Spec, .bcq-criteria.json, diff y resultados.

CARPETA DEL REPOSITORIO

labs/jornada/
06-revision-citada.md

ENTREGA

Criterio + fuente +
aplicación + resultado +
decisión.

Lab 06 · Petición para revisar un criterio

PROMPT DE TRABAJO

Commits base y del incremento · diff verificado con Git o GitHub · sigue el Entry de BCQuality y su skill de revisión · criterio: transacción implícita, sin Commit explícito · devuelve met / unmet / not evaluated y decisión. No es obligatorio encontrar defectos.

Núcleo del prompt · texto completo en la guía del Lab 06.

Puesta en común · Revisar la revisión

01 ¿La fuente sostiene realmente la recomendación?

02 ¿Qué parte del cambio se ha evaluado?

03 ¿Qué decisión tomaríais si falta evidencia?

Una referencia útil hace que la revisión se pueda contrastar.

Checkpoint humano · Aceptar el incremento

Comportamiento	La implementación satisface el contrato aprobado.
Calidad	Los hallazgos aplicables están resueltos o justificados.
Evidencia	Los resultados corresponden al código actual.
Límites	Las comprobaciones pendientes quedan explícitas.

Demo en Customer Card

Paso	Acción	Qué observar
1	Fijar WorkDate = 15/10/2026	Referencia común
2	Probar próxima fecha vacía, pasada, hoy y futura	Cuatro estados
3	Ejecutar Mark as reviewed	Última 15/10; próxima 14/11
4	Cerrar y reabrir la ficha	Persistencia
5	Repetir la acción ese día	Las fechas no vuelven a avanzar

Trabajo en Business Central · Recorrido funcional

10

MINUTOS APROX.

- 01 Probad los cuatro estados con la fecha de trabajo acordada.
- 02 Marcad revisado, cerrad y reabrid el cliente.
- 03 Repetid la acción y comprobad otros campos y permisos.

ENTREGA

Recorrido observado y cualquier diferencia con el contrato.

Puesta en común · Lo que vio el usuario

01 ¿La ficha refleja las reglas que aprobamos?

02 ¿Qué comprobación aportó información que no estaba en el diff?

03 ¿Aceptaría otra persona el incremento con vuestra entrega?

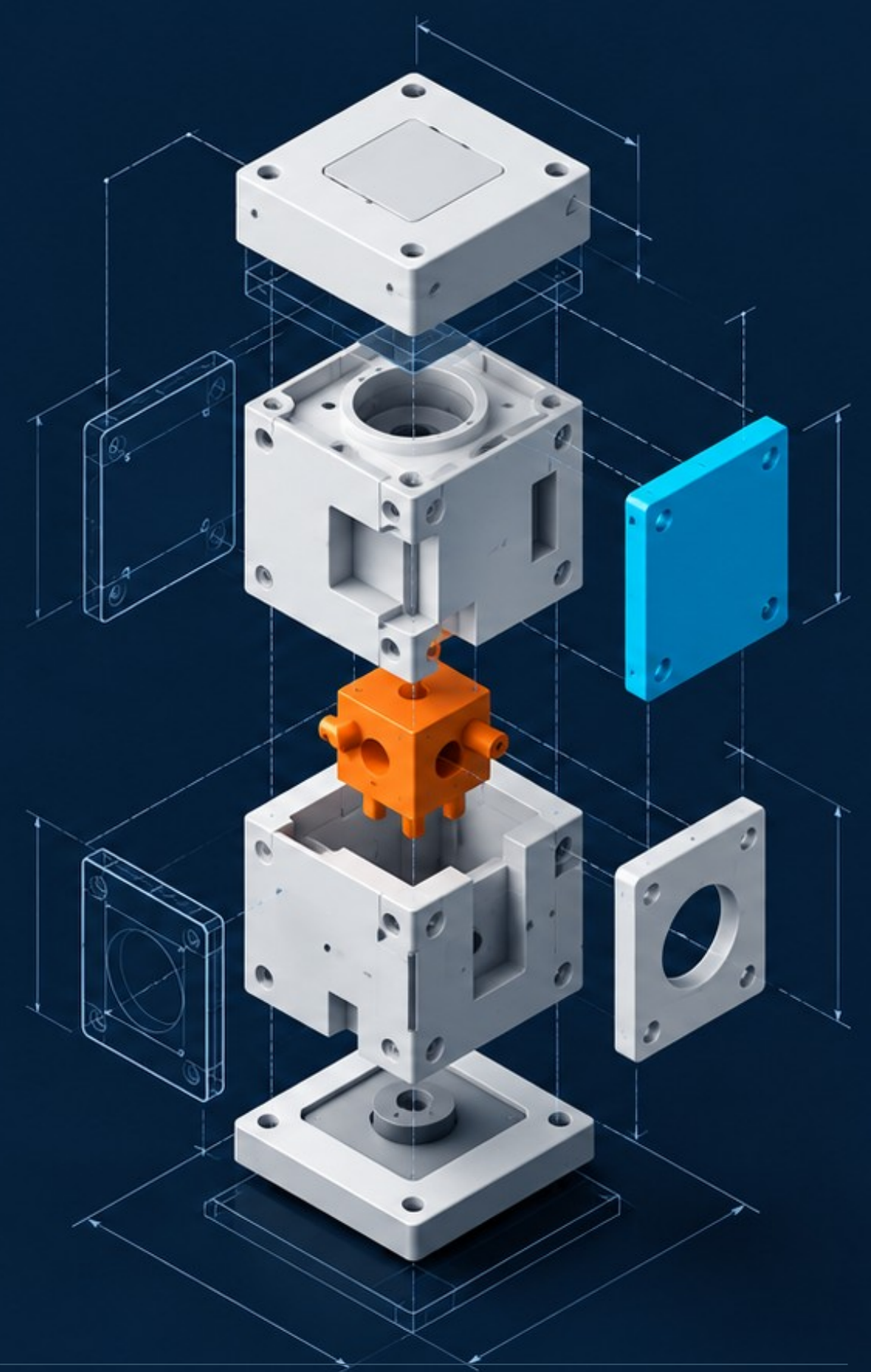
La aceptación reúne código, comportamiento y decisión.

PAUSA

Descanso

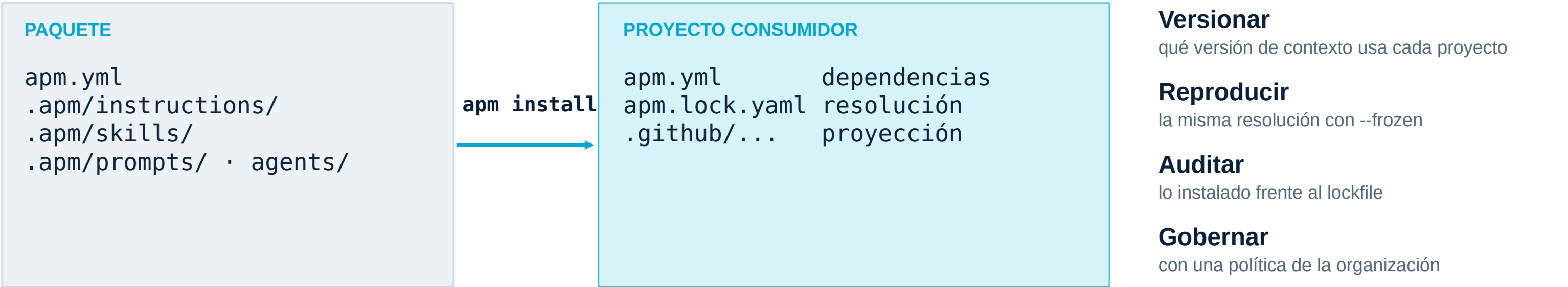
Volvemos a las 15:45

Guardad vuestro trabajo antes de salir.



¿Qué es APM?

Agent Package Manager: gestiona el contexto de los agentes como dependencias, igual que npm o pip gestionan código.



El contexto del equipo deja de copiarse a mano y pasa a ser una dependencia revisable.

APM: de qué se compone y cómo se usa

apm.yml Manifiesto: nombre, versión y dependencias.	.apm/ Primitivas del paquete: instrucciones, skills, prompts, agentes.	apm.lock.yml Resolución fijada en el consumidor.	--target Destino: copilot o claude.	apm-policy.yml Reglas de la organización (más adelante).
---	--	--	---	--

COMANDOS DEL TALLER · validados en el ensayo

apm install ../packages/october-workshop-primitives --target copilot
apm install --frozen --target copilot
apm audit

MÁS ADELANTE

apm pack y apm-policy.yml. No forman parte del recorrido validado.

Manifiesto para decidir, lockfile para repetir, auditoría para comprobar.

Qué distribuimos con APM

Paquete del equipo

Instrucciones y skills reutilizables.

Identidad y versión del contenido.

Dependencias declaradas.

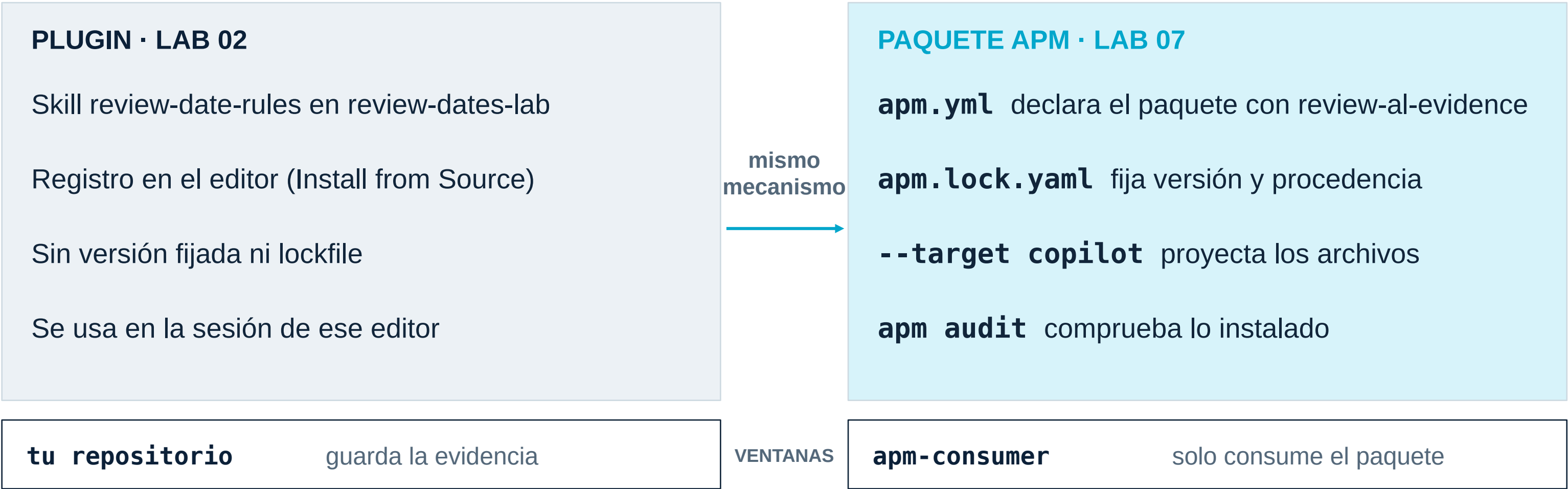
Proyecto consumidor

Manifiesto y resolución del paquete.

Archivos proyectados para el destino.

Actualizaciones que se pueden revisar.

Plugin y paquete APM



Otra forma de distribuir contexto: dependencia, versión y proyección explícitas.

APM y ALDC

¿Se puede distribuir ALDC con APM? Es un camino de adopción previsto: sus primitivas ya se organizan como paquete y el espacio reservado es javiarmesto/aldc, aún sin publicar.

EJEMPLO ILUSTRATIVO · NO VALIDADO

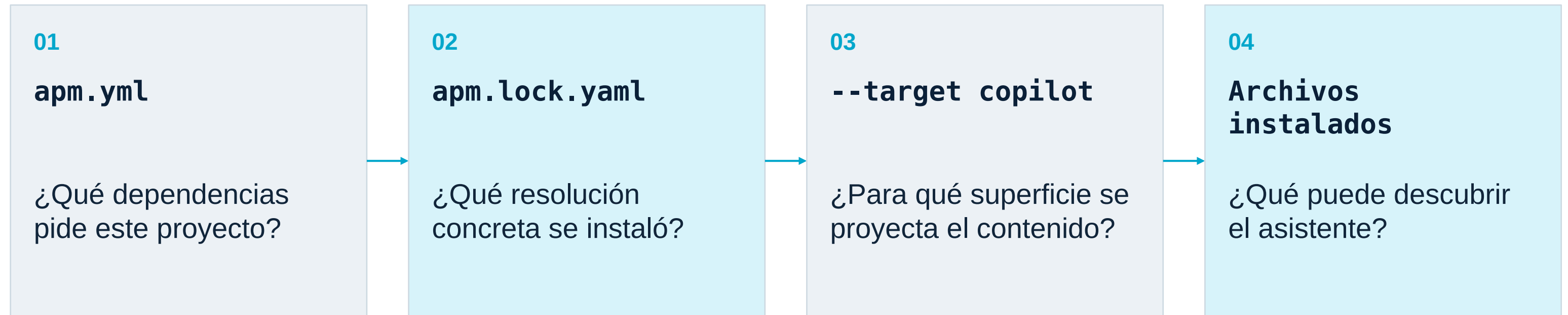
```
apm.yml · idea de dependencias

# ilustrativo: no es sintaxis validada
dependencias:
  javiarmesto/aldc      (previsto)
  <tu-org>/<tu-paquete>
  servidores MCP
```

- 01 **Inventariar** personalizaciones propias en un repo espejo
- 02 **Empaquetar** como paquete APM con espacio privado
- 03 **Declarar** ALDC + paquete propio + MCP en apm.yml
- 04 **Gobernar** apm-policy.yml: lista blanca, versiones, checksums

En esta edición ALDC se instala con la extensión 5.0.0; APM distribuye el contexto propio del equipo.

Manifiesto, lockfile y destino



apm install lee el manifiesto, fija la resolución en el lockfile y proyecta los archivos para Copilot.

Instalar en una carpeta consumidora

```
mkdir apm-consumer
cd apm-consumer

apm install ../packages/october-workshop-primitives --target copilot

apm install --frozen --target copilot
apm audit
```

Comprobación previa

apm --version debe responder antes de empezar.

Después

Abre apm-consumer en otra ventana e invoca review-al-evidence.

Desde la raíz de tu repositorio · destino Copilot

Lab 07 · Contexto como dependencia

Objetivo: Instalar y utilizar una skill desde un paquete del equipo.

20

MINUTOS APROX.

- 01 Instalad el paquete en una carpeta consumidora nueva.
- 02 Comparad manifiesto, lockfile y archivos instalados.
- 03 Repetid con --frozen y apm audit; invocad la skill en otra ventana.

PUNTO DE PARTIDA

Paquete local suministrado y APM disponible.

CARPETA DEL REPOSITORIO

labs/jornada/
07-contexto-como-dependencia.md

ENTREGA

Resolución del paquete + skill localizable y utilizada.

Una actualización también necesita revisión

Cambio	Qué comprobar
Instrucción del equipo	Qué comportamiento puede alterar
Nueva dependencia	Origen y capacidades que incorpora
Resolución del consumidor	Qué versión o revisión ha cambiado
Adopción	Quién acepta la actualización en el proyecto

Puesta en común · Reutilizar con control

01 ¿Qué cambió en el consumidor al instalar?

02 ¿Qué permite repetir el lockfile?

03 ¿Qué prueba que el agente usó la skill instalada?

Un equipo puede compartir contexto sin copiarlo a mano en cada proyecto.

Lab 08 · Entregar a otra persona

Objetivo: Conseguir que otra pareja pueda entender y comprobar el incremento.

10

MINUTOS APROX.

- 01 Intercambiad la entrega con otra pareja.
- 02 Reproducid un caso de fecha y la persistencia.
- 03 Aceptad o devolved indicando exactamente qué falta.

PUNTO DE PARTIDA

Spec aprobada, cambio, resultados y recorrido funcional.

CARPETA DEL REPOSITORIO

labs/jornada/
08-entregar-a-otra-persona.md

ENTREGA

Decisión de otra persona.
Un autoensayo no es aceptación.

El paquete de entrega del incremento

Pieza	Debe permitir
Requisito y spec	Entender el comportamiento acordado
Diff final	Identificar el cambio realizado
Resultados	Saber qué código y casos se comprobaron
Revisión	Entender criterios y hallazgos tratados
Recorrido de aceptación	Repetir la comprobación en la ficha

Diseñar un flujo para vuestro equipo

Elegir	Un cambio pequeño y frecuente del backlog.
Separar	Decisiones humanas y trabajo delegable.
Definir	Entradas, entregas y comprobaciones por rol.
Reducir	Quitar pasos que no mejoran el resultado.

Plantilla de diseño del flujo

Decisión	Vuestra respuesta
Cambio concreto	¿Qué resultado necesita el usuario?
Contrato	¿Qué debe quedar escrito antes del código?
Roles	¿Qué responsabilidades conviene separar?
Checkpoint	¿Dónde decide una persona?
Evidencia	¿Qué prueba o revisión permite aceptar?

Trabajo en parejas · Vuestro primer flujo

5

MINUTOS APROX.

- 01 Elegid un cambio real y acotadlo.
- 02 Escribid roles, entregas y un checkpoint.
- 03 Indicad qué señal usaréis para saber si ayuda.

ENTREGA

Propuesta de un experimento pequeño para el equipo.

Puesta en común · Qué os llevaréis al equipo

01 ¿Qué cambio habéis elegido?

02 ¿Qué decisión seguirá en manos de una persona?

03 ¿Qué paso habéis decidido no añadir?

Un flujo proporcionado y una forma concreta de evaluarlo.

Lo que hemos practicado

Objetivo	Resultado de la jornada
Orquestar con ALDC	Roles, delegación y retorno de revisión
Especificar antes de implementar	Arquitectura, spec y aceptación
Aplicar contexto reutilizable	Instrucciones, skill y plugin
Mantener control humano	Aprobación del contrato y del incremento
Distribuir con APM	Paquete instalado en un consumidor
Revisar con BCQuality	Criterio citado y aplicación al diff

Cuándo la estructura deja de ayudar

01 ¿Qué coordinación ahorró retrabajo?

02 ¿Qué contexto resultó redundante?

03 ¿Qué comprobación aportó más información?

Conservad los pasos que mejoran una decisión o una entrega.

Material para continuar

aldc-workshop-lab Starter, contrato, guías de los labs, preflight y evidencias.

ALDC Roles, flujos y documentación de uso.

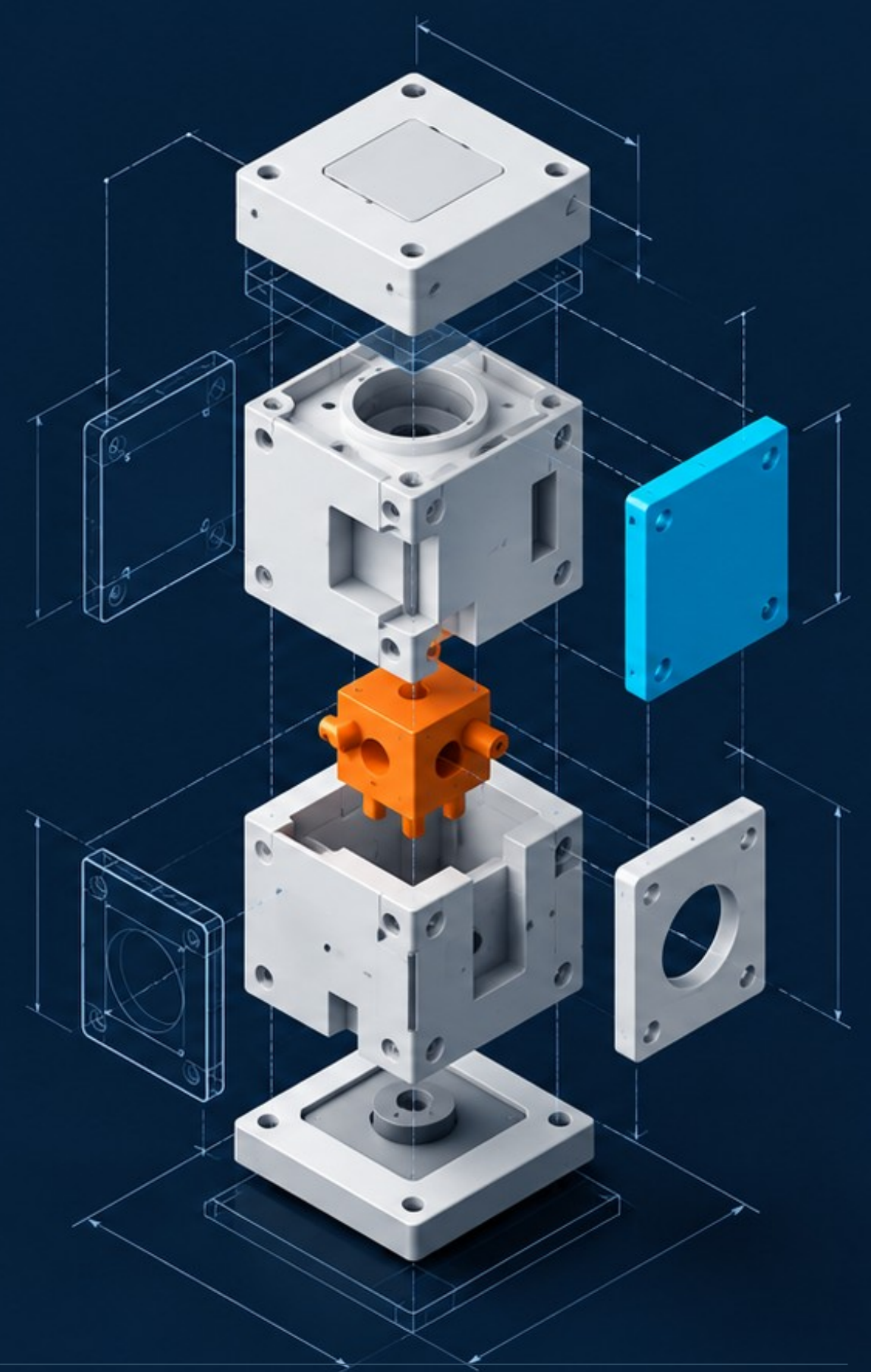
Recursos de apoyo VS Code, AI Engineer Coach, BCQuality y APM.

github.com/javiarmesto/aldc-workshop-lab

Del requisito a una entrega que otra persona puede revisar

Contrato explícito. Trabajo observable.
Criterio para aceptar.

Roberto Corella · Javier Armesto



Ruta de consulta por laboratorio

Lab	Guía en aldc-workshop-lab
01 · Contrato y contexto	labs/jornada/01-contrato-y-contexto.md
02 · Plugin	labs/jornada/02-plugin-utilizable.md + templates/plugin
03 · Mejora observable	labs/jornada/03-mejora-observable.md
04 · Especificación	labs/jornada/04-requisito-a-especificacion.md
05 · Implementación	labs/jornada/05-incremento-implementado.md
06 · Revisión citada	labs/jornada/06-revision-citada.md
07 · APM	labs/jornada/07-contexto-como-dependencia.md + packages/
08 · Aceptación	labs/jornada/08-entregar-a-otra-persona.md

Recuperar el punto de partida

Situación	Acción
Workspace mezclado con materiales	Preparar otra copia aislada del starter
No compila	Leer el diagnóstico y revisar App, Test y símbolos
Una regla cambió durante el desarrollo	Actualizar la spec y volver a aprobar
El agente repite reparaciones	Detener, localizar el fallo y acotar la siguiente tarea
No hay resultados de ejecución	Registrar el pendiente y resolver la herramienta o entorno
Hace falta un punto de control	Checkpoint preparado por el instructor; no es un commit personal

Alternativa a APM · Profundizar en aceptación

Minutos	Actividad	Entrega
5	Explicar permisos, persistencia y repetición	Plan de comprobación
20	Repetir el recorrido con revisión cruzada	Resultados y hallazgos
10	Preparar la entrega y decidir	Aceptación o devolución

Referencia rápida · Estados y fechas

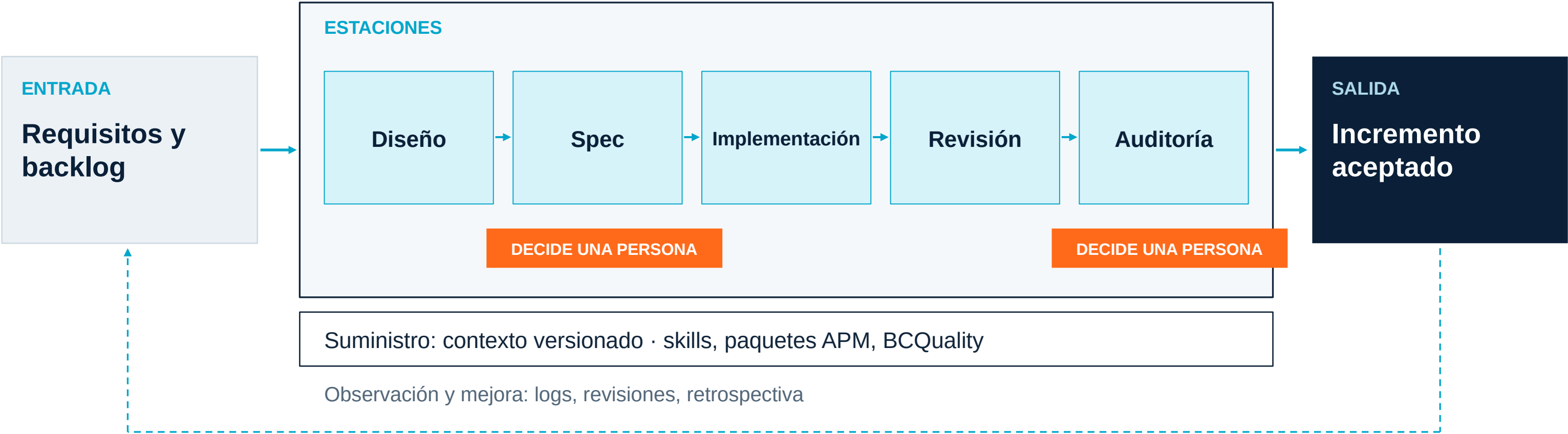
Regla	Resultado
Próxima vacía	Sin programar
Próxima < referencia	Vencida
Próxima = referencia	Revisar hoy
Próxima > referencia	Programada
Próxima = fecha de cierre (C)	Error de validación
Marcar revisado el 15/10/2026	Última 15/10/2026; próxima 14/11/2026
Repetir el 15/10/2026	Mismas fechas, sin acumulación

Recursos técnicos

Plantilla del taller	github.com/javiarmesto/aldc-workshop-lab
ALDC	github.com/javiarmesto/ALDC-AL-Development-Collection
Personalización de agentes en VS Code	code.visualstudio.com/docs/agent-customization/overview
AI Engineer Coach	github.com/microsoft/AI-Engineering-Coach
BCQuality	github.com/microsoft/BCQuality
Agent Package Manager	microsoft.github.io/apm/

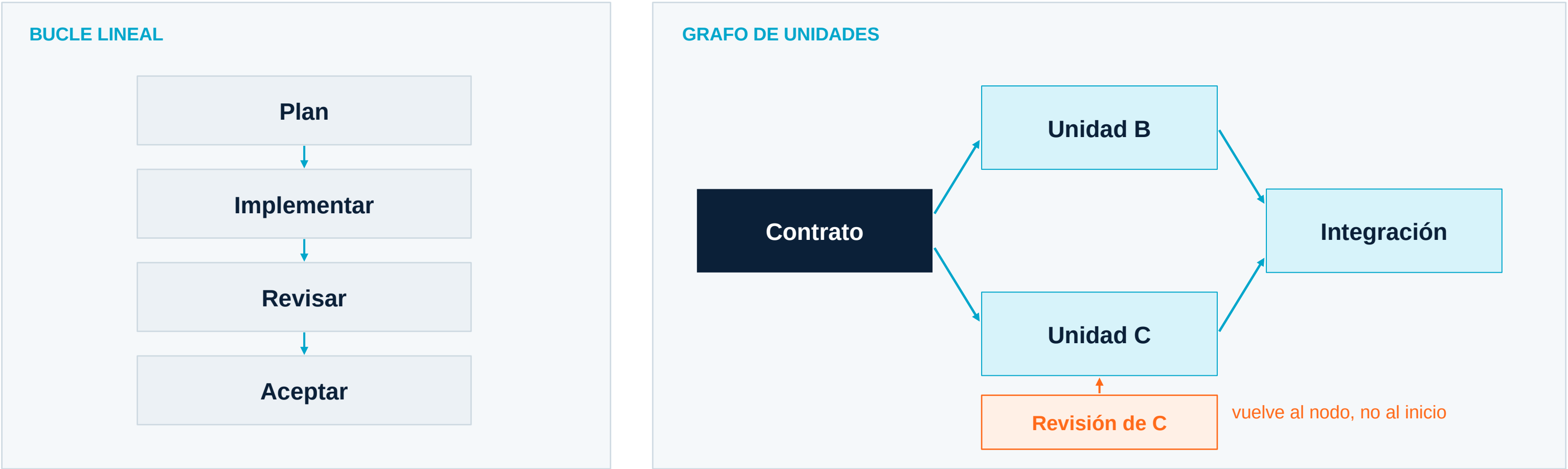
Hacia factorías de agentes

VISIÓN · NO ES EL TALLER



Hoy practicamos cada estación a la escala de un incremento.

Enfoque graph: cuando el trabajo no es una línea



Nodo: unidad con contrato propio · Arista: dato o contrato que se consume · B y C avanzan en paralelo

Paralelizar solo lo que no se consume entre sí; la revisión devuelve trabajo al nodo afectado.